

Fraud Risk Scoring Under a Fixed Review Budget

A temporal evaluation of credit card fraud data for a production-minded data science workflow

Giorgos Tzimas

June 2026

Abstract

This study evaluates a fraud-risk scoring workflow under severe class imbalance and a fixed review budget. The analysis uses the OpenML credit card fraud dataset, which contains 284,807 transactions and 492 confirmed fraud cases from a two-day period in September 2013 [1]. I trained two interpretable baseline models on the first 70% of transactions, selected the model with the best validation average precision, tuned a threshold to a 1% review budget, and evaluated it on the final 15% of transactions. A balanced random forest was selected. On the temporal test window, it reached a ROC AUC of 0.940 and average precision of 0.778. Reviewing the top 1% of transactions captured 44 of 52 fraud cases, a recall of 84.6%, with 11.1% precision after applying the validation threshold. The results support using the model as a triage layer for fraud operations, provided that it is paired with monitoring, calibration checks, delayed-label review, and careful production governance.

1 Introduction

Fraud risk modeling is a rare-event problem with operational consequences. A useful model is not simply the one with the best aggregate score. It needs to rank the riskiest cases well, support a realistic investigation budget, behave sensibly over time, and produce outputs that engineering and risk teams can monitor after deployment. A production-oriented workflow also has to make data quality, threshold choice, and model monitoring visible enough for risk teams to challenge and refine.

Credit card fraud data is a useful public proxy for that work. It is not identity verification data, and it does not include device, bureau, document, phone, address, or network signals. Still, it captures several conditions that matter in fraud and financial risk systems: severe class imbalance, a narrow review budget, temporal ordering, and the need to score many legitimate transactions while surfacing the small fraction that deserves attention. Prior fraud-detection work emphasizes the same practical concerns, including changing fraud behavior, delayed labels, active review, and the danger of evaluating models only under clean academic assumptions [3, 4].

This report answers one main question: can a compact, readable Python workflow create a fraud-risk score that captures a large share of fraud in a small review band on a later temporal holdout set? A secondary question is whether simple monitoring tables can reveal distribution movement that would matter before a score is trusted in production.

2 Research Questions and Hypotheses

The primary hypothesis is that a supervised risk model trained on earlier transactions will rank fraud cases well above random chance in a later test window. I evaluate that using average precision, ROC AUC, and fraud capture at fixed review budgets. Average precision and precision-recall curves receive more weight than accuracy because

the fraud class is less than 0.2% of all transactions, and precision-recall summaries are more informative under this kind of imbalance [7, 8].

The second hypothesis is operational: a threshold chosen from the validation set to approximate a 1% review budget will capture at least half of the fraud cases in the later test window while keeping alerts close to the intended review volume. This is the kind of target a risk operations team could discuss, tune, and adjust with business costs.

The third hypothesis is that monitoring will find measurable drift between the training and test windows. Fraud behavior and transaction mix can change over time, so a model that performs well once still needs a monitoring loop [11, 12].

3 Data

The dataset is OpenML `creditcard`, data_id 1597 [1]. It contains transactions made by European cardholders in September 2013. The source description reports 492 fraud cases among 284,807 transactions, or about 0.172% fraud. Features V1 through V28 are principal components created from confidential original variables. `Time` and `Amount` are retained in recognizable form, and `Class` is the binary fraud label. The dataset documentation asks users to cite the original unbalanced-classification work by Dal Pozzolo and colleagues [2].

I sorted transactions by `Time` and created a temporal split: the first 70% for training, the next 15% for validation, and the final 15% for testing. This is less convenient than a random split, but it is a better approximation of a deployed fraud system, where tomorrow’s transactions are scored with information learned yesterday.

Table 1: Temporal split summary

Split	Transactions	Fraud cases	Fraud rate	Mean amount
Train	199,364	384	0.193%	89.77
Validation	42,721	56	0.131%	97.52
Test	42,722	52	0.122%	72.54

Data quality checks found no missing cells. There were 1,081 duplicate rows after the anonymized PCA transformation. I retained them because the public data does not provide transaction identifiers, and repeated rows may represent legitimate repeated transaction signatures after anonymization. In a production system this check would be repeated with stable transaction IDs, customer IDs, merchant IDs, and ingestion timestamps.

4 Methods

The modeling workflow is intentionally simple. It reads the parquet file, engineers a small number of transparent features, fits two candidate models, evaluates the selected model on temporal splits, writes CSV outputs, creates plots, and stores scoring tables in a local SQLite database. The companion SQL file shows how the same feature construction could be expressed in a PostgreSQL-style warehouse table.

The engineered features were intentionally plain. `amount_log` was defined as $\log(1 + \text{Amount})$. `time_hours` measured elapsed time, `hour_of_day` represented a coarse daily cycle, and `amount_is_zero` flagged zero-amount transactions. All original V1 through V28 components, `Time`, `Amount`, and engineered features were used as candidate predictors.

I compared a class-weighted logistic regression and a class-weighted random forest. Logistic regression is a

useful rare-event baseline [10]. Random forests can capture nonlinear interactions without requiring a large amount of feature-specific modeling code [9]. The implementation used scikit-learn and pandas [13, 14].

Class imbalance was handled through class weighting rather than oversampling. Sampling methods such as SMOTE are important in the imbalanced-learning literature [6, 5], but for this project I wanted the score distribution to remain close to the original transaction mix. That choice is also easier to explain in a production review setting.

5 Statistical Analysis

Model selection used validation average precision. Final reporting used the temporal test split only. I reported ROC AUC for discrimination, average precision for rare-event ranking, and the Brier score for probabilistic error [15]. I also calculated review-budget metrics at 0.5%, 1%, 2%, and 5% of transactions. Those metrics are easy to connect to staffing and risk-operations tradeoffs.

The validation threshold was the 99th percentile of validation scores, corresponding to a 1% review target. That threshold was then applied unchanged to the test window. This avoids choosing the operating point after looking at the test labels.

For monitoring, I grouped test transactions into eight time buckets and tracked transaction count, fraud rate, average score, alert rate, and alerts. I also calculated population stability index (PSI) between train and test distributions for Amount, amount_log, hour_of_day, and V1 through V8. PSI is not a full concept-drift test, but it is a practical first screen for distribution movement.

6 Results

The balanced random forest had the best validation average precision and was selected for final evaluation. On the temporal test window, the selected model reached a ROC AUC of 0.940 and average precision of 0.778.

Table 2: Selected model performance

Split	Transactions	Fraud cases	ROC AUC	Avg. precision	Brier score
Train	199,364	384	1.000	0.984	0.000193
Validation	42,721	56	0.946	0.863	0.000388
Test	42,722	52	0.940	0.778	0.000458

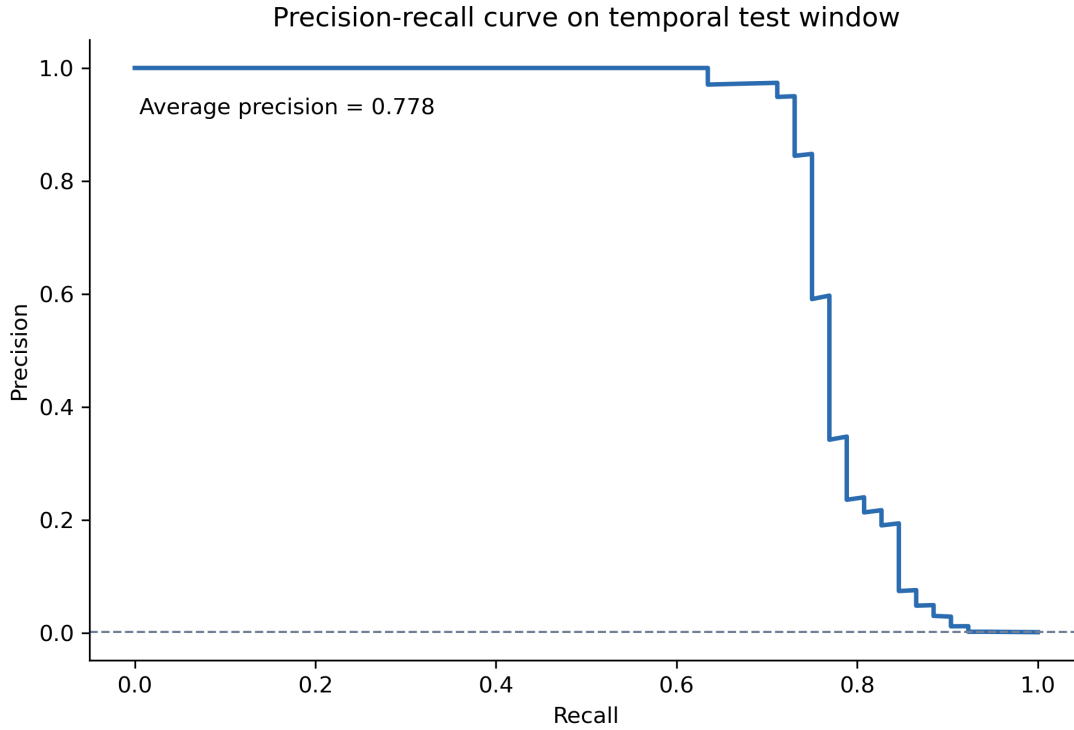


Figure 1: Precision-recall curve on the temporal test window

The review-budget results are the strongest part of the project. In the test set, the top 1% of scores contained 44 of 52 fraud cases. That is 84.6% fraud capture from 428 reviewed transactions, with a precision of 10.3%. Since the base fraud rate in the test window was only 0.122%, this is roughly 84.5 times the random-review baseline.

Table 3: Fraud capture at fixed review budgets on the test window

Review rate	Reviewed	Fraud found	Precision	Fraud capture
0.5%	214	43	20.1%	82.7%
1.0%	428	44	10.3%	84.6%
2.0%	855	45	5.3%	86.5%
5.0%	2,137	47	2.2%	90.4%

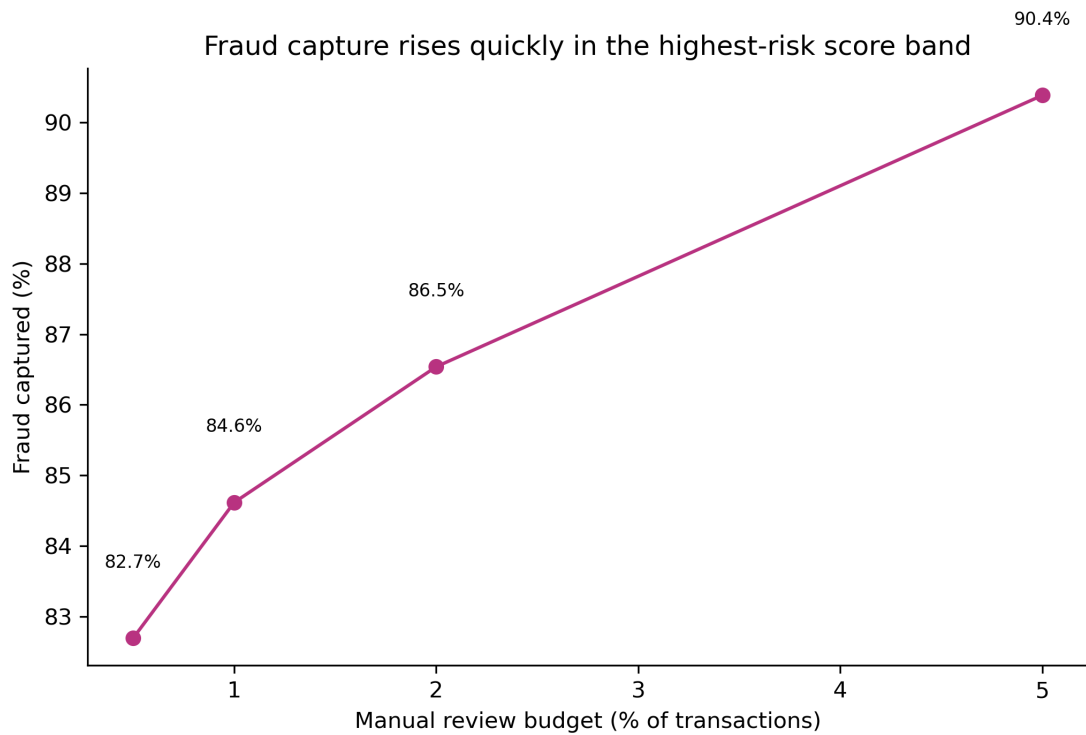


Figure 2: Fraud capture by manual review budget

Applying the validation-tuned 1% threshold to the test set produced 395 alerts out of 42,722 transactions, or a 0.925% alert rate. It identified 44 true positives, missed 8 fraud cases, and produced 351 false positives. The resulting recall was 84.6% and precision was 11.1%. This supports the operational hypothesis: the threshold stayed close to the intended review budget and captured well above half of the later fraud cases.

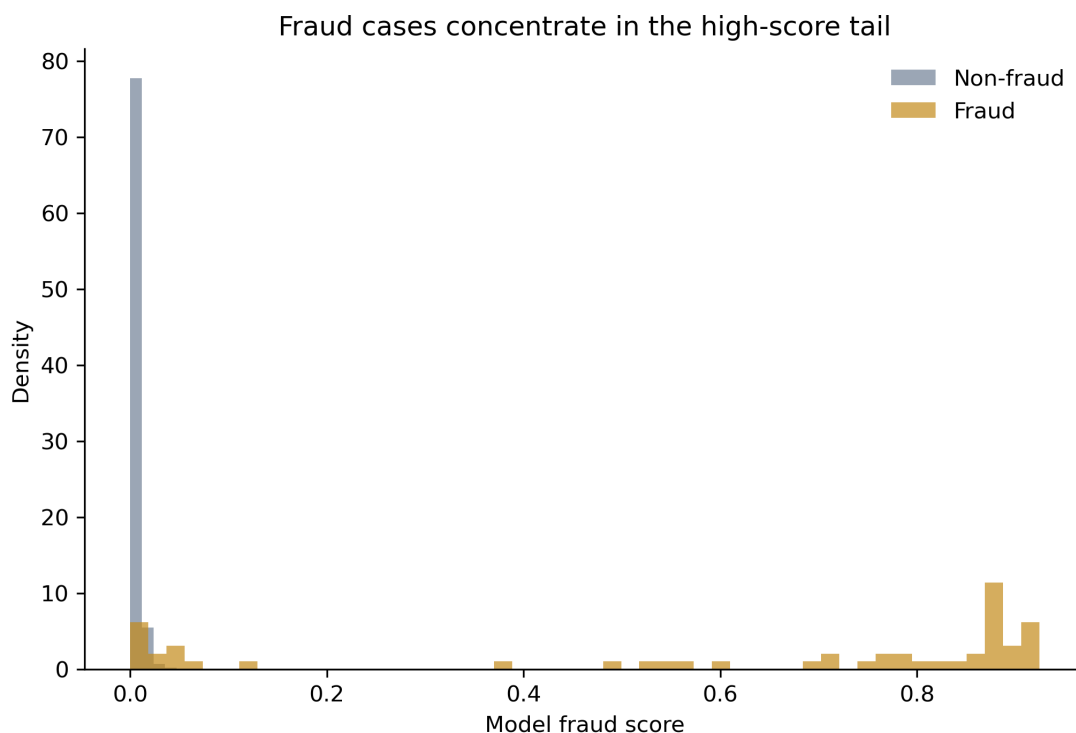


Figure 3: Model score distribution by true class

The highest random forest feature importances were mostly principal components, which limits domain interpretation because the public dataset anonymized the original fields. In a richer fraud-risk product, this same table would be more useful if the features represented device velocity, address consistency, phone age, document signals, bureau attributes, or entity-graph features.

7 Monitoring and Production Readiness

The monitoring output shows nontrivial movement across the later time buckets. Average score and alert rate do not stay completely flat, which is expected for a two-day fraud dataset but still useful as a production warning pattern.

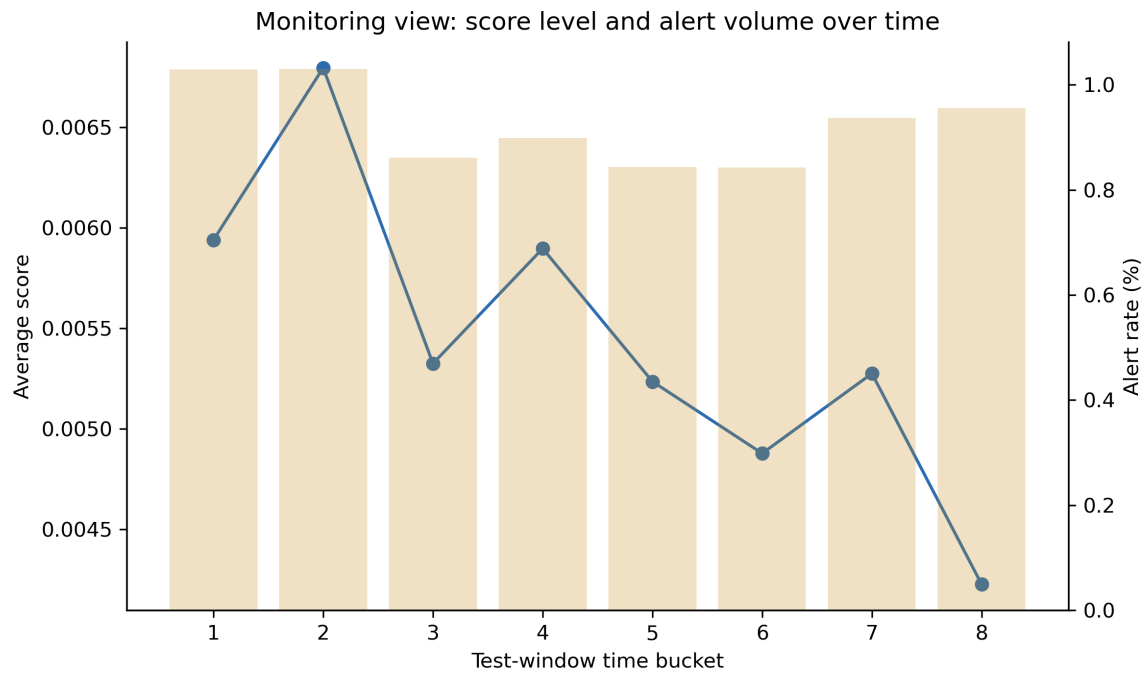


Figure 4: Monitoring view of average score and alert volume over time

PSI also flagged major distribution movement for `hour_of_day`, `V1`, and `V3`, plus smaller movement in `V4` and `V5`. I would not treat these PSI values as proof that the fraud concept changed, but I would treat them as a reason to inspect score stability, approval impact, and label arrival before relying on a fixed threshold for long.

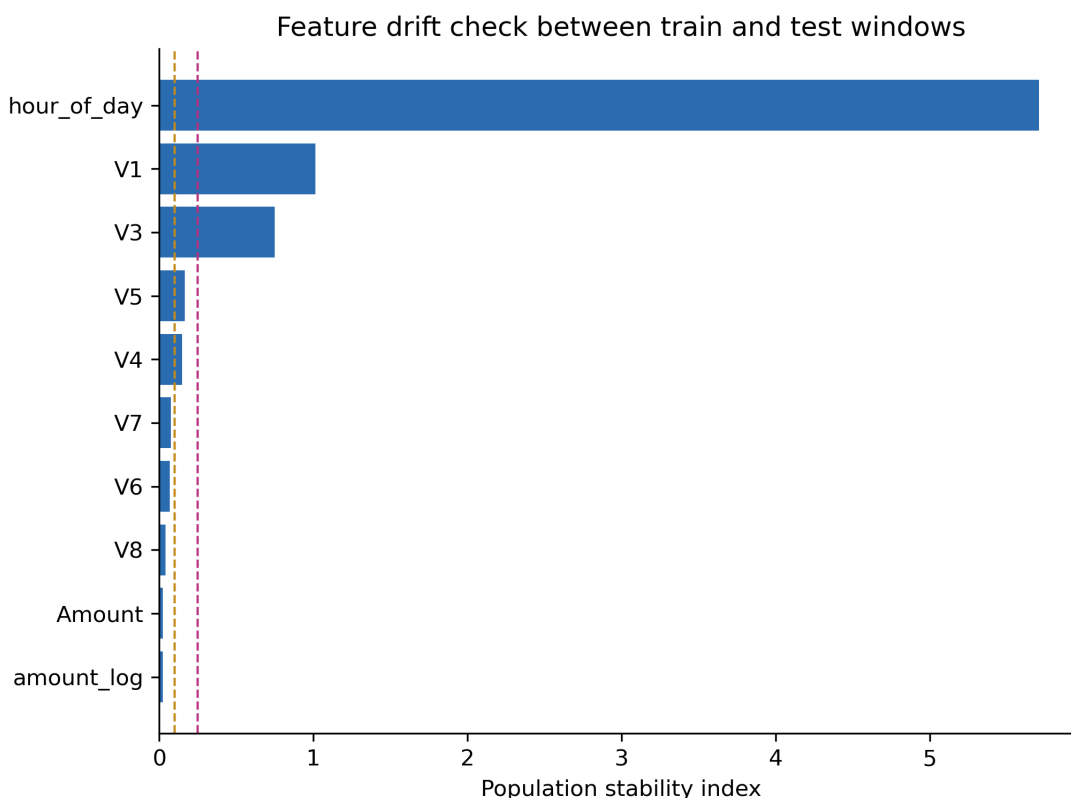


Figure 5: Feature drift screen using population stability index

The project also includes a local SQLite scoring database and a PostgreSQL-style feature scoring template. In a cloud-oriented production version, the raw parquet transaction data could land in object storage, feature tables could be maintained in a relational or analytical warehouse, a scheduled compute job could produce scores, and monitoring tables could feed a dashboard or alerting workflow. This project does not claim a cloud deployment. It is a production-style design sketch supported by local artifacts.

8 Discussion

The evidence supports the main hypothesis. The selected model ranked rare fraud cases far above random review in a later temporal test set. More importantly, it did so under a small review budget. A top-1% strategy captured most fraud cases while keeping the alert volume small enough to be operationally plausible.

The model should not be read as a final fraud decisioning system. It is a triage model. In practice, identity and financial-risk systems need stronger features, careful latency budgets, explainability for partner conversations, data acquisition decisions, and review loops that feed labels back into the model. A model that catches fraud at 1% review in a historical public dataset can still fail if the transaction mix shifts, fraudsters adapt, labels arrive late, or a threshold causes too much friction for legitimate applicants.

The project is useful because it shows the full workflow in a compact form: data quality checks, temporal splitting, feature engineering, model comparison, validation-based thresholding, fixed-budget reporting, SQL feature logic, local scoring tables, and monitoring outputs. That is more useful than a notebook that only reports an accuracy score.

9 Limitations

The public dataset is anonymized and old. The PCA features prevent direct business interpretation. The data covers only two days, so the temporal test is useful but limited. There are no customer, merchant, device, identity, bureau, address, phone, email, or document-verification signals. The label process is not visible, which matters because real fraud labels can be delayed and biased by investigation strategy. Finally, the model was not deployed to AWS. The AWS portion is represented through design-compatible artifacts rather than a running cloud service.

10 Conclusion

A compact Python fraud-risk pipeline can produce a useful ranking model under severe class imbalance. In this project, the selected balanced random forest captured 84.6% of test fraud cases at a validation-tuned review threshold that alerted less than 1% of transactions. The monitoring outputs also showed that score and feature distributions move enough to justify production checks. The project demonstrates Python modeling, SQL-style feature construction, rare-event evaluation, data quality checks, thresholding, monitoring, and a cloud-aware path from analysis to production.

References

- [1] OpenML. creditcard, data_id 1597. 2015. <https://www.openml.org/api/v1/json/data/1597>
- [2] Dal Pozzolo, A., Caelen, O., Johnson, R. A., and Bontempi, G. Calibrating Probability with Undersampling for Unbalanced Classification. 2015 IEEE Symposium on Computational Intelligence and Data Mining.
- [3] Dal Pozzolo, A., Caelen, O., Le Borgne, Y.-A., Waterschoot, S., and Bontempi, G. Learned Lessons in Credit Card Fraud Detection from a Practitioner Perspective. *Expert Systems with Applications*, 41(10), 4915–4928, 2014.
- [4] Carcillo, F., Le Borgne, Y.-A., Caelen, O., and Bontempi, G. Streaming Active Learning Strategies for Real-Life Credit Card Fraud Detection: Assessment and Visualization. *International Journal of Data Science and Analytics*, 5, 285–300, 2018.
- [5] He, H., and Garcia, E. A. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284, 2009.
- [6] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357, 2002.
- [7] Davis, J., and Goadrich, M. The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233–240, 2006.
- [8] Saito, T., and Rehmsmeier, M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3), e0118432, 2015.
- [9] Breiman, L. Random Forests. *Machine Learning*, 45, 5–32, 2001.
- [10] King, G., and Zeng, L. Logistic Regression in Rare Events Data. *Political Analysis*, 9(2), 137–163, 2001.
- [11] Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. A Survey on Concept Drift Adaptation. *ACM Computing Surveys*, 46(4), 44:1–44:37, 2014.

- [12] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28, 2015.
- [13] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830, 2011.
- [14] McKinney, W. Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61, 2010.
- [15] Brier, G. W. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review*, 78(1), 1–3, 1950.